Lahore University of Management Sciences
School of Science and Engineering, Department of Artificial Intelligence
AI651 – Deep Learning for Space, Time and Graphs
Fall 2026

Assignment 2

Date Assigned: Wednesday, Oct. 7th, 2026 **Date Due:** Monday, Oct. 26th, 2026 11:55pm

---

Instructions

- While discussion with your peers is strongly encouraged, complete the assignment independently. In order to gain the maximum benefit, try to perform solo brainstorming before any consultations. The thought process behind searching for the correct approach all on your own will be invaluable.
- **Late Submission Policy.** Late solutions will be accepted with a 25% penalty for one day after the deadline and a 50% penalty for the second day. No solution will be accepted after this.
- The Professor and TAs reserve the right to hold a viva with any student if a discrepancy is found.
- Students must submit one PDF report containing their answers, results, and findings through the LMS Assignments Tab.
- **Code.** Create a GitHub repository, provide its link on the first page of the report, and set it to PUBLIC at submission time.
- Students are welcome to use generative AI tools as partners in their learning journey. However, they are required to explicitly mention the prompt, its output, and edits made at the end of the report. For details, please refer to the AI policy listed on the course outline.

Question 1 — River Forecasting Proposal

Before you design: a 20-minute companion notebook

`Question1/Assignment_2.ipynb` walks through a few small experiments on toy sensor networks. It covers what happens when a forecast lacks information it needs, how S, T and joint ST blocks build space–time paths, how far a stack can reach, and what attention buys and what it costs. Every model is **already trained**: you do not train anything and you do not need a GPU (setup is in `Question1/readme.md`). You run the cells, inspect forecasts and reach and attention maps, debug a failing forecaster, and think about the design decisions behind each result.
These are the same decisions you will face below. Twenty minutes spent watching small models succeed and fail should make your design easier to reason about and to defend. The notebook is **optional and ungraded**, and you submit nothing from it. Its experiments may or may not prove directly useful for this question, but we recommend working through it first.

A government water-management authority is commissioning a forecasting system to support river monitoring and flood preparedness. Your team has been invited to pitch a solution to a committee of government decision-makers and technical reviewers.

The service: every ten minutes, predict the water level **one hour ahead at each of the four main-river stations, S1–S4**.



Project setting. Arrows indicate downstream flow; the illustration is not to scale.

The reservoir releases water into the main river. A monitored tributary joins between S1 and S2; an unmonitored tributary joins between S2 and S3. Rainfall, tributary inflows, reservoir operations, and upstream conditions can affect downstream water levels. Stations are unevenly spaced, and travel times can change with river conditions.

Available information

- Historical water levels at S1–S4 and tributary station T1, recorded every ten minutes.
- Historical rainfall readings at R1 and R2, with their timestamps and gauge locations.
- Recorded reservoir releases and future release schedules announced before each forecast is issued, with their announcement and effective times.
- Station locations, river connectivity, and distances along the river.

Precise travel times and inflows from the unmonitored tributary are not supplied. Future rainfall and future water-level measurements are not available at issuance. Additional information may be requested from the authority, but its availability is not guaranteed. Where details such as record length, reporting delays, or measurement quality are unspecified, state reasonable assumptions.

Response Q1 *assessed*

Design your forecasting system. Use these six report headings in this order. Answer directly and concisely. Choose any defensible architecture; there is no word or page limit.

1. **Architecture diagram.** Draw the architecture first: show each data source, the main processing blocks, source combination, and forecasts for S1–S4. Label main tensor dimensions and any alignment, projection, or reshaping needed to combine inputs.
2. **Assumptions.** List each assumption separately as A1, A2, etc. Explain why each is reasonable and how it affects your design. Distinguish assumptions from supplied facts. Cover unspecified data conditions and the river dynamics your model assumes.
3. **Forecasting task and data specification.** Define issuance time, your history window, the one-hour horizon, and outputs for S1–S4. In one table, list each input and output, its dimensions, axis meanings (including any batch axis), and timestamps or time coverage. Explain alignment across locations and sampling times, and identify information available at issuance.
4. **Model computations and design justification.** Define notation and give core equations for the main blocks, forecast mapping, and training objective. Justify major choices and explain how upstream conditions, tributaries, rainfall, and reservoir releases influence each forecast. Distinguish physical assumptions from learned mappings. Refer to your diagram and assumptions; exhaustive intermediate tensor dimensions are not required.
5. **Operation and exceptional conditions.** Explain separately how you handle (i) missing observations, (ii) delayed observations, (iii) announced future releases, and (iv) differences between scheduled and actual releases. Justify feasibility of running every ten minutes; no implementation or benchmark is required.
6. **Uncertainty, limitations, and additional information.** List uncertainties and limitations, including the unmonitored tributary and changing travel times. Explain each one's forecast impact and your response, or what remains unresolved. List each additional information request, how it would help, and whether the system remains usable without it.

No dataset, implementation, or training run is required. A separate physical simulator or probabilistic model is optional; explain your treatment of physical relationships and uncertainty.

Question 2 — Motion or Memory?

A video predictor that does well on its test set has not necessarily learned *how things move*. It may just have learned *what the training videos usually look like*. You can tell these apart by changing the video in ways the motion itself should not care about (flipping it, rotating it, playing it faster) and checking whether the predictions still make sense.

In this question you will train a ConvLSTM forecaster on Moving MNIST, stress-test it with these transformations, work out why it fails where it does, and then build a model that holds up. No starter code is provided; everything you need is defined below.

Number your tables and figures as in this handout (Table 1, Figure 3, ...) and answer the questions under their numbers (Q2.1, Q2.2, ...), so we can match each answer to the results it refers to. Report errors to three significant figures, and use scientific notation for values below 10^{-3}.

1. Setup

Data

Load **Moving MNIST** (Srivastava et al., 2015) with `torchvision`:

```
torchvision.datasets.MovingMNIST(root=..., split=None, download=True)
```

This returns all 10,000 complete videos, each with 20 frames of 64×64 grayscale pixels showing two handwritten digits that move in straight lines at constant speed. Digits bounce off the walls and pass through each other when they overlap.

Do not use the dataset's built-in `split="train"` or `split="test"` options: they split the *frames* of each video, not the videos. Instead, split the videos manually by dataset index: videos 0–7,999 for training, 8,000–8,999 for validation, and 9,000–9,999 for testing. Convert frames to `float32` and divide by 255 so that intensities lie in $[0, 1]$. Frames are numbered 1–20 throughout this question.

Unless stated otherwise, every model-performance number and every prediction example you report must use the 1,000 test videos. The exceptions are Table 1 (synthetic tensors), Figure 1 (training videos), and parameter counts and timings. The validation split may be used for implementation checks, for monitoring training, and for choosing the design of model D (What Can Data Fix?); never use the test split to choose between candidate designs. In this question the test split serves as the evaluation and analysis set.

Forecasting task

The model sees 5 frames and predicts the next 5. At normal speed ($1\times$), a window starting at frame t is

$$X = (x_t, \ldots, x_{t+4}), \qquad Y = (x_{t+5}, \ldots, x_{t+9}). \tag{1}$$

The index $h = 1, \ldots, 5$ always counts *forecast horizons* (the first, second, ... predicted frame), not video frames. Unless stated otherwise, evaluate with $t = 1$ (input frames 1–5, target frames 6–10).

Transformations

Each transformation is applied to every frame of both the input X and the true future Y. For a pixel $x[i, j]$ (row i, column j, indices 0–63):

- **Mirror** M: $(Mx)[i, j] = x[i, 63 - j]$. The digits change appearance (a mirrored 2 is not a 2).
- **Rotation** R: 90° anticlockwise, i.e. `torch.rot90(x, k=1, dims=(-2,-1))`. The digits change appearance even more.
- **Speed-up** ($k\times$): keep every k-th frame, so the digits move further between the frames the model sees.

Mirror and rotation, together with their combinations, give the 8 symmetries of the square, $G = \{ R^a,\ MR^a : a = 0, 1, 2, 3 \}$, where $(MR^a)(x) := M(R^a(x))$ (rotate first, then mirror). R^a has inverse R^{-a}, and each MR^a is its own inverse.

Windows at different speeds

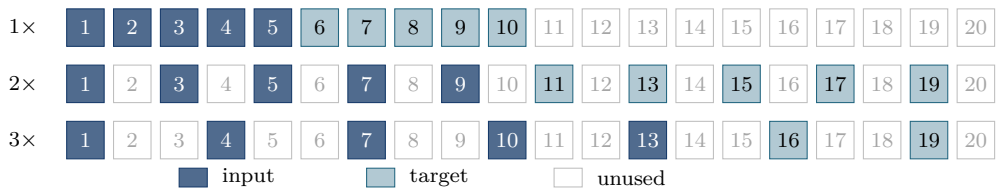
For speed $k \in \{1, 2, 3\}$, a window starting at frame t is

$$X = (x_t, x_{t+k}, \ldots, x_{t+4k}), \qquad Y = (x_{t+5k}, \ldots, x_{t+9k}), \tag{2}$$

where only target frames that exist (frame number ≤ 20) are kept. Two rules decide which start frames are allowed:

- **Normal-speed training (models A and B):** $t \in \{1, \ldots, 11\}$, so all five target frames exist.
- **Speed-augmented training (models C and D):** all five input frames and at least two target frames must exist. This gives $t \in \{1, \ldots, 14\}$ at $1\times$, $t \in \{1, \ldots, 8\}$ at $2\times$ and $t \in \{1, 2\}$ at $3\times$.

With $t = 1$, the frames used are:



Every model always generates five predictions. When fewer than five target frames exist, the extra predictions are still generated but are left out of the loss and of every metric. For example, at $3\times$ with $t = 1$ the model makes five predictions, and only the first two (compared with frames 16 and 19) are scored. For the same reason, **whenever you compare across speeds, use only the first two forecast horizons**.

Checking your code

Before going further, check that your transformation and window functions do exactly what is defined above, using the same functions your experiments will call. Use two test videos of shape $(20, 1, 64, 64)$:

- a **random video** x: `torch.manual_seed(0); x = torch.rand(20, 1, 64, 64)`;
- a **marker video** p: all zeros except a single bright pixel, $p[7, 0, 10, 20] = 1$ (frame 7, row 10, column 20, indices from 0).

Mirror and rotation only rearrange pixels, so checks (a)–(c) must hold *exactly*: report the maximum absolute difference between the two sides, which must be 0, and `torch.equal` must return `True`. Checks (d)–(e) catch mistakes that (a)–(c) cannot, such as flipping the wrong axis, rotating clockwise, or reordering frames. For (f)–(h), run your window function with $t = 1$ on a video whose frame f is filled with the value f, and read off which frames it returns. Report the results as **Table 1**, in this format, with the last column filled in:

	Check	Expected	Yours
(a)	$\max\|M(M(x)) - x\|$	0	
(b)	$\max\|R^4(x) - x\|$	0	
(c)	$\max\|MRM(x) - R^{-1}(x)\|$	0	
(d)	bright pixel of $M(p)$ (frame, row, column)	$(7, 10, 43)$	
(e)	bright pixel of $R(p)$ (frame, row, column)	$(7, 43, 10)$	
(f)	1× input / target frames	1–5 / 6–10	
(g)	2× input / target frames	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	
(h)	3× input / target frames	1, 4, 7, 10, 13 / 16, 19	

If any row does not match, fix your code before continuing: every later result depends on these functions.

Is the training data already symmetric?

Constant-velocity motion with elastic bounces in a square box is still valid motion after a mirror or a 90° rotation, and faster motion is valid too. So a model that has really learned the motion rule should cope with all three transformations. Whether the transformed inputs are also *unfamiliar* to the model depends on the training data, so look at that first.

Estimate dense optical flow on the **first 50 training videos**, between every pair of consecutive frames (x_s, x_{s+1}), $s = 1, \ldots, 19$, giving $50 \times 19 = 950$ frame pairs. Compute the flow on frames converted to `uint8` (multiply by 255) with OpenCV's Farneback method and these settings:

```
cv2.calcOpticalFlowFarneback(prev, next, None, 0.5, 3, 15, 3, 5, 1.2, 0)
```

Use exactly this method and these settings for all reported results. For each pixel the flow gives a displacement (d_x, d_y), where d_x is along columns and d_y along rows (rows increase *downward*). Keep only pixels whose intensity on the $[0, 1]$ scale exceeds 0.2 at the same location in both x_s and x_{s+1} (equivalently, above 51 on the `uint8` scale), and whose flow magnitude $\sqrt{d_x^2 + d_y^2}$ exceeds 0.5 pixels. Pool the kept pixels from all 950 pairs.

Figure 1 should show, for these pooled pixels:

(a) a histogram of flow **direction** $\theta = \text{atan2}(-d_y, d_x)$, in degrees on $[0°, 360°)$ with 18 bins of 20°, so that 0° is rightward and 90° is upward on screen, with a horizontal line at 1/18 (the level of a perfectly uniform distribution);

(b) a histogram of flow **speed** $\sqrt{d_x^2 + d_y^2}$ in pixels per frame, with the median marked by a vertical line.

Plot both as the fraction of kept pixels (not raw counts), and state the median speed and the number of pixels kept in the caption.

Q2.1 Based on Figure 1(a), would mirrored and rotated trajectories already be familiar to a model trained on this data? (Hint: how would the direction histogram change if every training video were mirrored? If every video were rotated by 90°?) From Figure 1(b), what is a typical speed, and what do 2× and 3× correspond to in pixels per frame? Optical flow is unreliable where digits overlap; could that bias your histograms enough to change your answer?

Q2.2 For each transformation, does it change the motion statistics, the digit appearance, or both? Why does this make the three of them useful as a controlled experiment?

2. Baseline: ConvLSTM

Architecture

The baseline, **model A**, follows the ConvLSTM of Shi et al. (2015) but *without* its peephole terms; implement exactly the equations below.

1. Fold each 4×4 pixel patch into channels with `torch.nn.functional.pixel_unshuffle(x, 4)`, so a $1 \times 64 \times 64$ frame becomes $16 \times 16 \times 16$.
2. Two ConvLSTM layers, each with 64 hidden channels and 3×3 kernels (stride 1, zero padding 1). At step τ, layer 1 takes the folded frame and layer 2 takes layer 1's new hidden state $h_\tau^{(1)}$. Hidden and cell states start at zero for every video.
3. A 1×1 convolution maps layer 2's hidden state from 64 to 16 channels, then `pixel_shuffle(·, 4)` and a sigmoid give back a $1 \times 64 \times 64$ frame.

Each ConvLSTM layer $\ell \in \{1, 2\}$, with input $u_\tau^{(\ell)}$, computes

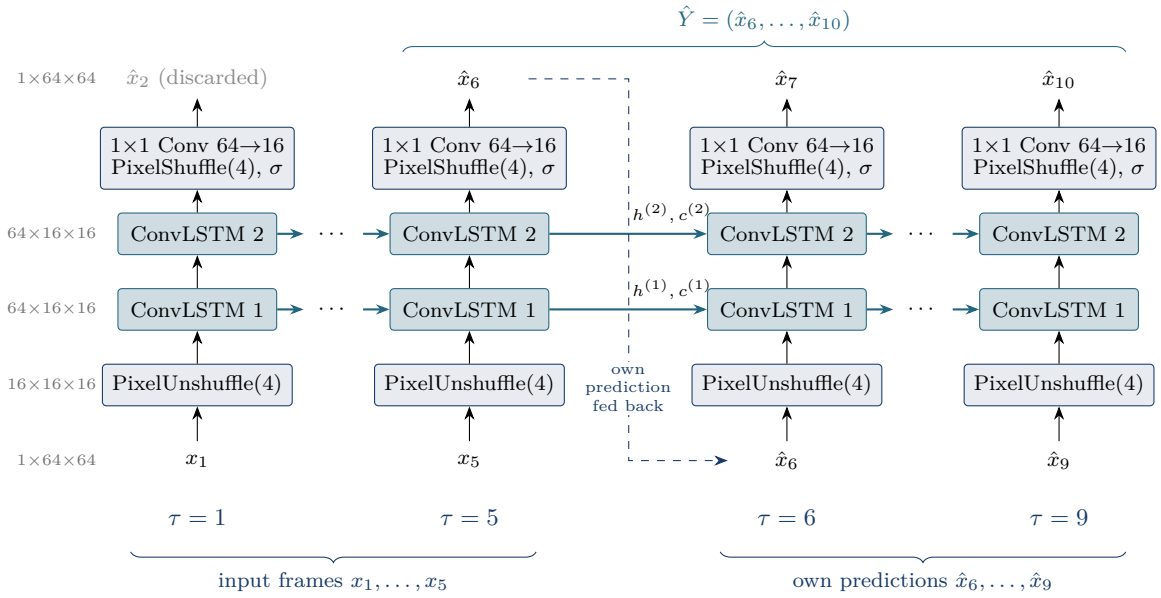
$$[\, i,\, f,\, o,\, g\,] = \mathrm{Conv}_{3\times3}^{(\ell)}([u_\tau^{(\ell)},\, h_{\tau-1}^{(\ell)}]), \tag{3}$$

$$c_\tau^{(\ell)} = \sigma(f) \odot c_{\tau-1}^{(\ell)} + \sigma(i) \odot \tanh(g), \qquad h_\tau^{(\ell)} = \sigma(o) \odot \tanh(c_\tau^{(\ell)}), \tag{4}$$

where $[\cdot, \cdot]$ is channel concatenation and one convolution produces all four gates. The input is $u_\tau^{(1)}$ = the folded frame and $u_\tau^{(2)} = h_\tau^{(1)}$. Each layer has its own weights.

All convolutions use a learnable bias (`bias=True`) and PyTorch's default initialisation. Do not add normalisation, dropout, residual connections or any other layers to models A–C. As a check, model A has exactly **480,784** trainable parameters.

Rollout. The network runs for 9 steps. At step τ it takes a frame u_τ and outputs a prediction $\hat{x}_{\tau+1}$. For $\tau = 1, \ldots, 5$, u_τ is the τ-th input frame; for $\tau = 6, \ldots, 9$ it is the network's own previous prediction $\hat{x}_\tau$. The outputs of steps 5–9 are $\hat{Y}$; the outputs of steps 1–4 are thrown away. Use this rollout during **both training and evaluation**. Do not use teacher forcing: true future frames are never fed to the network, and gradients flow through the fed-back predictions. The diagram below shows the rollout.



Model A unrolled over its 9 steps (steps 2–4 and 7–8 omitted), shown for a 1× window with $t = 1$. Every column uses the *same* weights. Teal arrows carry layer ℓ's hidden and cell state $(h^{(\ell)}, c^{(\ell)})$ to the next step. From step 6 on, the input is the network's own previous prediction. Only the outputs of steps 5–9 form $\hat{Y}$. Grey labels give tensor shapes (channels × height × width).

Training and evaluation

Every learned model in this question is trained the same way, unless its row in the model table of What Can Data Fix? says otherwise:

- **Optimiser and schedule:** Adam with learning rate 10^{-3} and no schedule, batch size 32, 30 epochs; keep the final-epoch weights. One epoch is one pass over the 8,000 training videos (in shuffled order) with one randomly sampled window per video, drawn under the rules in Setup.
- **Loss:** squared error between prediction and target, averaged over the pixels and the *available* target frames of each video, then averaged over the videos in the batch. Every training video therefore contributes equally, whether it has two or five target frames.
- **Seeds:** train every learned model with exactly two seeds, 0 and 1. Each seed controls initialisation, shuffling, window sampling and augmentation sampling. In every table, report each entry as *mean (seed 0, seed 1)*. Unless stated otherwise, use the **seed-0 model** for all figures, GIFs and per-video histograms.
- **Timing:** report the hardware used. Training time is the mean wall-clock time per epoch over all 30 epochs, excluding the one-time dataset download. Inference time is the wall-clock time to predict all 1,000 original (1×, $t = 1$) test videos at batch size 32 under `model.eval()` and `torch.no_grad()`, starting after each batch is on the model's device. For symmetrised models (The Final Model), include all eight transformations, forward passes, inverse transformations and the averaging.

You will also need two predictors that need no training: **Blank** (all-zero frames) and **Copy-last** (repeat the last input frame). They are deterministic, so report a single value for them. For a test

video, let MSE_h be the mean squared error over the 64×64 pixels of forecast horizon h, and define

$$E^{(5)} = \frac{1}{5} \sum_{h=1}^{5} \mathrm{MSE}_h, \qquad E^{(2)} = \frac{1}{2} \sum_{h=1}^{2} \mathrm{MSE}_h, \tag{5}$$

each averaged over the 1,000 test videos.

How far can information travel?

A 3×3 convolution only mixes a position with its immediate neighbours, so information moves a limited distance each time one is applied. In a ConvLSTM some of these convolutions act on the input and some on the recurrent state, at every time step, and patch folding changes how many pixels "one position" is. We call the largest distance (in original pixels) that information can travel through the network in one time step its **reach**. If a digit moves well beyond this distance between frames, the network cannot connect the digit's old position, held in its recurrent state, to its new position within a single update, so tracking it becomes much harder.

Use these conventions so that everyone measures the same quantity:

- measure distance centre-to-centre in the original 64×64 image; one cell of the folded 16×16 grid corresponds to 4 original pixels, and the 4×4 patch width itself is *not* added;
- only spatial convolutions with a neighbourhood larger than 1×1 move information; the 1×1 output convolution, `pixel_unshuffle` and `pixel_shuffle` do not;
- consider every path within one step τ, from the new input u_τ or from the previous states $h^{(1)}_{\tau-1}, h^{(2)}_{\tau-1}$ to the prediction $\hat{x}_{\tau+1}$, and report the longest.

Report the following:

- **Table 2:** $\mathrm{MSE}_1, \ldots, \mathrm{MSE}_5$, $E^{(5)}$, and trainable parameter count for model A, Blank and Copy-last.
- **Figure 2:** input, prediction and ground truth for three test videos, at least one of them with overlapping digits.

Q2.3 Estimate model A's reach in original pixels per time step, using the conventions above. Walk through your reasoning from the patch size, kernel sizes, number of layers and the recurrence. There is no formula to look up for this.

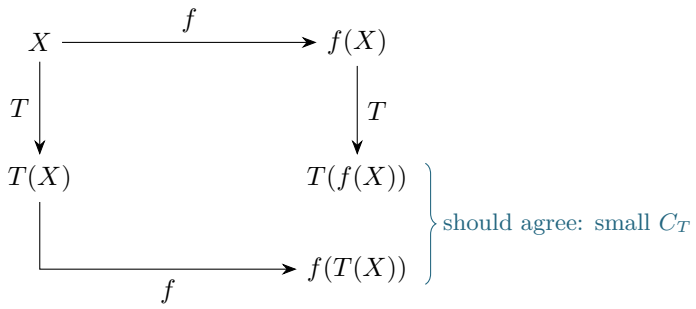
Q2.4 Using Table 2 and Figure 2, how does model A compare with the two reference predictors, and how does its error grow over the five forecast horizons? What happens to its predictions when digits overlap?

Q2.5 Before you run any experiment in Testing the Baseline, write down your predictions: will model A pass the mirror test? The rotation test? At what speed should it break down, given your reach estimate and Figure 1? **Include this answer in your report exactly as you first wrote it**, even if it turns out to be wrong.

3. Testing the Baseline

Mirror and rotation

Let f be a trained forecaster and $T \in \{M, R\}$. If f has learned a rule that carries over to the transformed scene, then predicting from the transformed input should give the same result as transforming the original prediction, i.e. the diagram below should commute.



We measure this with the **consistency gap** C_T, and measure correctness with the **transformed error** E_T:

$$C_T = \mathrm{MSE}\left(f(T(X)),\, T(f(X))\right), \qquad E_T = \mathrm{MSE}\left(f(T(X)),\, T(Y)\right). \tag{6}$$

Both are computed exactly like $E^{(5)}$: at $1\times$ with $t = 1$, the squared error is averaged over the pixels and the five forecast horizons of each video, then over the 1,000 test videos. A model can be perfectly consistent and still wrong, which is why both are needed.

Report the following:

- **Table 3:** $E^{(5)}$, C_M, E_M, C_R and E_R for model A, Blank and Copy-last.
- **Figure 3:** the two test videos with the largest per-video C_M (with $T = M$) and the two with the largest per-video C_R (with $T = R$), for the seed-0 model A. For each video, show four rows of ten frames, with the five input frames in columns 1–5 and five future frames in columns 6–10:

Row	Columns 1–5	Columns 6–10
(1)	X	$f(X)$
(2)	$T(X)$	$T(Y)$
(3)	$T(X)$	$f(T(X))$
(4)	$T(X)$	$T(f(X))$

  If the same video is among the top two for both transformations, show it twice, once for each transformation. Also save each panel as an animated GIF in your repository that steps through the five forecast horizons while keeping the same four rows, and give the GIF paths in the caption.
- **Figure 4:** histograms of the per-video C_M and C_R of the seed-0 model A over the test set, with the videos from Figure 3 marked.

Q2.6 Blank and Copy-last are *exactly* consistent under mirror and rotation. What does this say about reading C_T on its own, and why does E_T always have to be reported next to it?

Q2.7 Pick one video in Figure 3 where rows 3 and 4 disagree and describe precisely how the model's account of the motion changes. For example, a digit goes the wrong way, gets lost, bounces too early or not at all, or blurs into something unrecognisable.

Q2.8 Is rotation harder than mirroring for model A? Using your answers to Q2.1 and Q2.2, argue whether the difference comes from the motion or from the digits' appearance.

Faster motion

Now play the test videos at 2× and 3× (with $t = 1$) and see whether model A keeps up.

- **Table 4:** model A's $E^{(2)}$ at 1×, 2× and 3×, and its $E^{(5)}$ at 1× and 2×, with Copy-last for comparison.
- **Figure 5:** for one test video, the five input frames and the first two predicted and true future frames at 1×, 2× and 3× (seed-0 model A). Choose a video whose digits do not overlap in any of the displayed true frames.

Q2.9 At what speed do model A's predictions fall apart, and does this agree with the reach you estimated in Q2.3? If not, what did your estimate miss? Describe what the failure in Figure 5 looks like (blur, a digit that stalls, a duplicated digit, a digit that vanishes) and what each of these suggests the model is doing.

Q2.10 From these results alone, can you tell whether the speed failure happens because fast motion never appeared in training, or because it is beyond the model's reach? Propose an experiment that would tell the two apart.

4. What Can Data Fix?

By now there are three possible explanations for model A's failures. Its motion rule might be tied to *what the digits look like* (appearance); it might never have seen this kind of motion in training (unfamiliar motion); or the motion might be faster than information can travel through the network (architectural reach). To separate these, train three more models under the protocol in Training and evaluation. Models B and C each change one thing relative to model A; model D changes only the architecture relative to model C.

Model	Architecture	Training speeds	Symmetry augmentation
A	baseline (model A)	1× only	none
B	baseline	1× only	g drawn uniformly from G
C	baseline	k drawn uniformly from $\{1, 2, 3\}$	none
D	larger reach, your design (see below)	k drawn uniformly from $\{1, 2, 3\}$	none
D+B (optional)	same as D	k drawn uniformly from $\{1, 2, 3\}$	g drawn uniformly from G

- **Symmetry augmentation:** for each training window, draw g uniformly from the 8 elements of G (including the identity R^0) and apply the same g to the input and the target.
- **Speed augmentation:** for each training window, draw k uniformly from $\{1, 2, 3\}$, then draw t uniformly from the allowed starts for that k (Setup).
- **Model D:** a convolutional recurrent network of your design with at most 1.5× model A's trainable parameters, i.e. at most 721,176. Model D **must have a strictly larger one-step reach than**

model A, under the conventions in How far can information travel?; you demonstrate this in Q2.12 with the same calculation you used in Q2.3. Larger kernels, dilated convolutions, a bigger patch size or an extra downsampling stage are all reasonable options; attention is not allowed. Keep everything else identical to model A: input and output shapes, the 9-step rollout without teacher forcing, and the training protocol. You may compare candidate designs on the *validation* split only.

Q2.11 **Before training B–D**, predict in at most three sentences which model will fix which of the failures you found in Testing the Baseline. As with Q2.5, include this unedited in your report.

Then report **Table 5**: for models A–D, $E^{(5)}$ at 1×; C_M, E_M, C_R, E_R; $E^{(2)}$ at 1×, 2× and 3×; trainable parameter count; training time per epoch; and inference time.

Q2.12 Describe model D and estimate its reach the same way you did in Q2.3, showing that it is strictly larger than model A's.

Q2.13 Label each failure from Testing the Baseline as appearance, unfamiliar motion, or architectural reach, citing the comparison in Table 5 that supports each label. In particular, if model C still does badly at 3× while model D does well, what does that tell you, and why can't more training data carry information further than the architecture allows?

Q2.14 Did any of these changes cost accuracy on the original 1× clips? If so, was it worth it?

5. The Final Model

Symmetry augmentation *encourages* consistency but does not guarantee it. Alternatively, you can build the symmetry into the forecaster directly. For any trained model f, the **symmetrised forecaster** is

$$\bar{f}(X) = \frac{1}{8} \sum_{g \in G} g^{-1}(f(g(X))), \tag{7}$$

that is, transform the input by each symmetry, predict, undo the transformation, and average. This needs no retraining. Show (a short argument is enough) that $\bar{f}$ has $C_M = C_R = 0$ exactly for any f. Your implementation should give zero up to floating-point error; report these values in Table 6.

Your final model is model D wrapped as a symmetrised forecaster. If Table 5 supports it, you may instead train D+B (see the model table) and wrap that; say which you chose and why.

- **Table 6:** the Table 5 columns for symmetrised B, symmetrised D, and your final model if it is different. In the training-time column, report the training time of the underlying learned model (B for symmetrised B, D for symmetrised D, D+B for symmetrised D+B); symmetrisation itself needs no additional training. Give C_M and C_R of the wrapped models in scientific notation.
- **Figure 6:** (a) Figure 3 repeated with your final model on exactly the same videos; (b) Figure 5 repeated with your final model on exactly the same video. Save the GIFs for (a) in your repository, as for Figure 3.

Q2.15 Compare symmetry that is learned (model B) with symmetry that is built in (the symmetrised models) in terms of consistency, accuracy on original and transformed clips, and inference cost. Which one is exact, and why? Averaging eight predictions can also blur them; when would you expect built-in symmetry to hurt accuracy, and did it here?

Q2.16 Go back to the video you discussed in Q2.7. Does the final model now get it right (Figure 6), and which change fixed which part of the failure?

Q2.17 **Final verdict** (150 words max). Did your final model learn motion? What evidence would convince a skeptic, and what is one test it still has not passed?

Looking ahead (ungraded)

Two ideas from this question carry over to the next assignment. First, a convolution applies the same filter everywhere, so it is consistent under shifts by construction; the symmetrised forecaster does the same for mirror and rotation. On a graph, the symmetry that matters is relabelling the nodes, and graph neural networks are built to respect it. Second, reach on a graph is measured in hops: k message-passing layers can only combine information from nodes within k hops. Recall the teaser at the end of Assignment 1, where a disturbance took 12 samples to travel from station 3 to station 4: a graph model can only use station 3's warning if station 3 is within its reach, in hops as well as in time.

Submission notes

- Include Tables 1–6 and Figures 1–6. Tables 2–6 and Figures 2–6 use the test split; Table 1 uses the synthetic tensors and Figure 1 the training videos.
- Your answers to Q2.5 and Q2.11 must be the versions written before running the corresponding experiments. Commit each to your repository before running those experiments; we may check the commit times.
- Always report a consistency gap together with its transformed error, and report every learned model for both seeds (0 and 1).
- When you make a claim about a result, say which model, transformation and speed it refers to.

References

Shi, X., Chen, Z., Wang, H., Yeung, D.-Y., Wong, W.-K., and Woo, W. (2015). *Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting.* NeurIPS.

Srivastava, N., Mansimov, E., and Salakhutdinov, R. (2015). *Unsupervised Learning of Video Representations using LSTMs.* ICML.